Broker-Based Communications

Overview

Peer-to-Peer Is Problematic for IoT

In a peer-to-peer model, every device must know the network address of every other device it needs to communicate with, and each device must accept incoming connections. This creates several problems:

- **Scalability:** With N devices, the number of potential connections grows as N×(N-1). In a class of 30 students, each with a microcontroller and a laptop, that's 3,480 potential point-to-point connections to manage.
- **NAT and firewall traversal:** Most networks place devices behind Network Address Translation (NAT) or firewalls that block inbound connections. Peer-to-peer communication requires open inbound ports, which enterprise networks almost never allow for IoT devices.
- **Device discovery:** Devices need a way to find each other. IP addresses can change when devices reconnect to WiFi, making static addressing unreliable. Peer-to-peer systems require additional discovery mechanisms.
- **Reliability:** If one device is temporarily offline, messages sent to it are lost. There is no intermediary to store messages and deliver them when the device reconnects.

As a result, peer-to-peer communication is rarely used in IoT deployments.

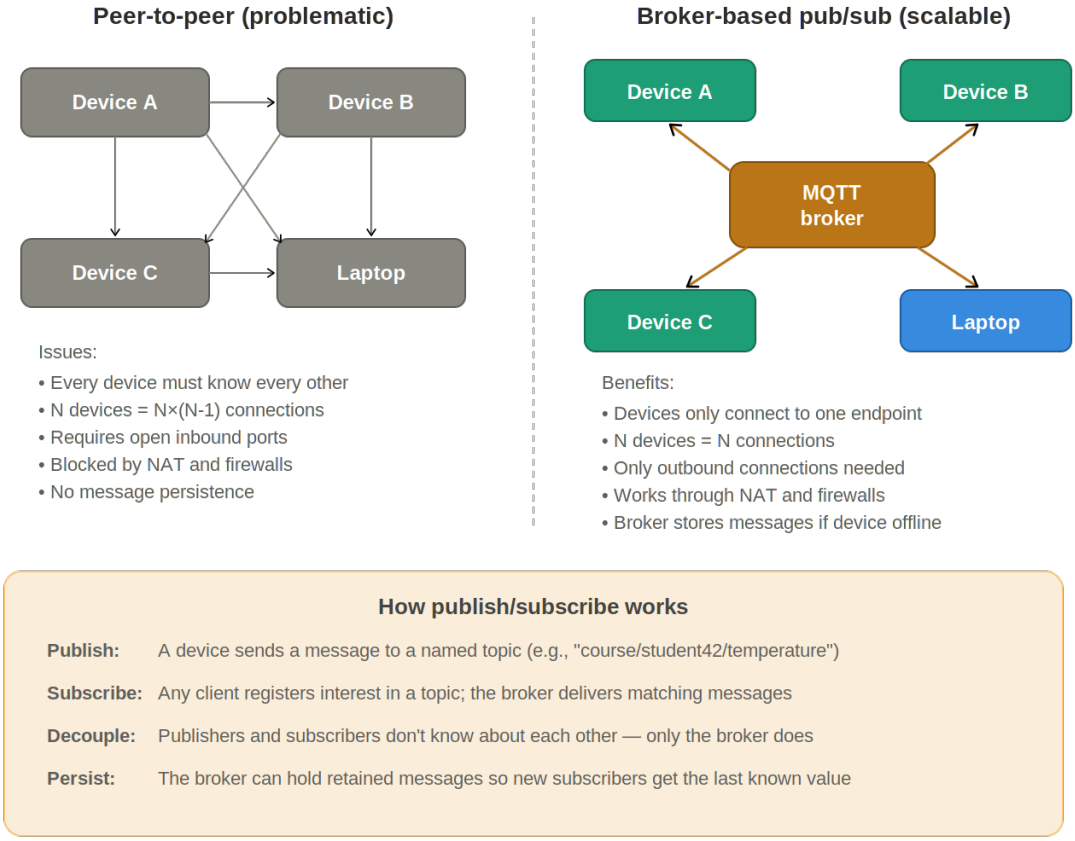


Figure 1: Peer-to-peer communication creates a mesh of connections and requires open ports. Broker-based pub/sub reduces complexity to a star topology with only outbound connections.

Broker-Based Communications: Publish/Subscribe with MQTT

The standard solution in IoT is the publish/subscribe (pub/sub) model, mediated by a central message broker. Instead of devices talking directly to each other, every device connects to a single broker. The dominant pub/sub message protocol for IoT is **MQTT (Message Queuing Telemetry Transport)**. MQTT was designed for constrained devices and unreliable networks. Key characteristics include: minimal packet overhead (as little as 2 bytes per message header), built-in keep-alive and reconnection logic, quality-of-service levels to guarantee delivery, retained messages for state synchronization, and a lightweight client library that runs comfortably on microcontrollers with limited memory. Devices that produce data publish MQTT messages to specified topics. Devices that want to receive data subscribe to those MQTT topics. The broker handles all the routing; publishers and subscribers never need to know about each other. The broker can be implemented using a dedicated custom server, for example using the open-source *Eclipse Mosquitto* broker or Python *amqtt* module, or using a third-party cloud-based broker such as HiveMQ (the ENME441 option), RabbitMQ, EMQ X, or AWS IoT Core.
Regardless of broker implementation, this architecture solves every problem that peer-to-peer creates:

- **Scalability:** N devices require only N connections (one per device to the broker), not N×(N-1).
- **Firewall-friendly:** Every device initiates an outbound connection to the broker. No inbound ports need to be open.
- **No discovery needed:** Devices only need to know the broker's address — a single, stable endpoint.
- **Message persistence:** The broker can retain the last message on each topic, so a device that reconnects can immediately receive the most recent state.

We will use HiveMQ as a cloud MQTT broker. HiveMQ provides a free tier that supports up to 100 concurrent connections (sufficient for our needs) with full TLS encryption. Because it is a managed cloud service, there is no server to maintain, no software to install, and no infrastructure to manage. The broker requires TLS (encrypted connections), which is important because MQTT messages traverse the public internet. The MicroPython ssl module supports TLS natively, so no additional hardware or software is required. MQTT topics are hierarchical strings separated by forward slashes. We will use the following convention to keep each user's data organized and prevent collisions:

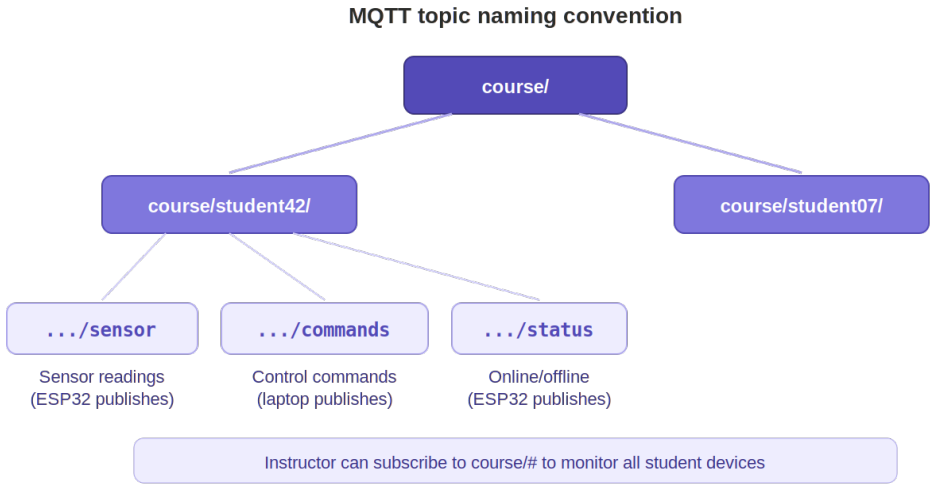


Figure 2: Topic naming convention using terpmail usernames as unique student IDs to prevent collisions.

Each student's topics are namespaced under their terpmail username, ensuring no interference between students. The three standard subtopics provided in the initial sample code (to be discussed below) are:

- **enme441/your_terpmail_username/sensor** — ESP32 publishes sensor or state data to this topic. A client running on a laptop subscribes to this topic to read and/or display the published data.
- **enme441/your_terpmail_username/commands** — Client publishes control commands (e.g., turn on LED) to this topic. The ESP32 subscribes to this topic and acts on any incoming commands.
- **enme441/your_terpmail_username/status** — the ESP32 publishes its connection status (online/offline) to this topic, using MQTT's Last Will and Testament feature to automatically publish an "offline" message if the device disconnects unexpectedly.

WiFi and HiveMQ Setup

The campus eduroam network relies on WPA2/WPA3-Enterprise authentication (802.1X), which requires a device to support entering individual username/password credentials and to run a supplicant capable of certificate-based EAP authentication. Most IoT devices lack the interface, firmware, or protocol support needed to complete this handshake, since they're typically designed only for simpler WPA2-Personal (pre-shared key / PSK) networks. In our case, ESP32 microcontrollers running MicroPython connect via the built-in network module, whose WLAN driver only implements standard WPA2-Personal. Because eduroam also federates across institutions and ties each connection to a personal login, it isn't well-suited to headless devices that need a stable, non-expiring connection independent of any one user's credentials.

We will instead use the umd-iot network for connecting our IoT devices to the Internet. The umd-iot network exists specifically to solve the eduroam issues for IoT devices. It uses a simpler authentication scheme (MAC-address registration) that the ESP32 can support, while still keeping IoT traffic isolated from the main campus network for security purposes. However, the umd-iot network isolates devices from each other and from eduroam, preventing us from implementing peer-to-peer interfacing for devices on this network. This isolation has three critical implications for ENME441:

- **Cross-subnet isolation:** Devices on umd-iot cannot communicate with devices on eduroam, and vice versa. A laptop on eduroam cannot directly reach an ESP32 on umd-iot.
- **Peer-to-peer isolation:** Devices on umd-iot are also isolated from each other. One ESP32 cannot communicate directly with another ESP32, even though both are on the same network.
- **Inbound connection isolation:** the umd-iot network blocks all inbound connections. This means that even if you knew the IP address of an ESP32, you could not SSH into it, send HTTP requests to it, or establish any new connection to it from outside.

Due to these constraints, you cannot open a terminal on your laptop and connect directly to your microcontroller over the campus WiFi. Protocols that require direct device-to-device connections (e.g. opening a raw TCP socket) will not work. We will overcome this constraint by using a cloud MQTT broker (HiveMQ) acting as an intermediary. Both your laptop and the ESP32 make outbound connections to the broker, and the broker relays messages between them:

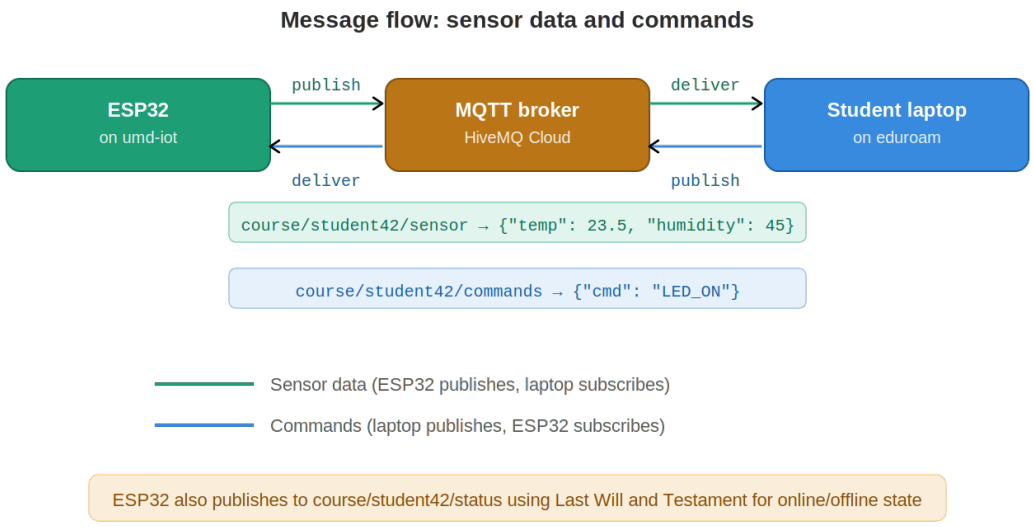


Figure 3: Brokered message flow. The ESP32 publishes sensor data; the laptop publishes commands. The broker routes messages in both directions.

The ESP32 initiates an outbound TCP connection to the cloud broker when it boots. The umd-iot network allows outbound internet connections, so this succeeds. Once the TCP connection is established, it remains open through periodic keep-alive pings (the MQTT protocol handles this automatically). Because TCP connections are inherently bidirectional, the broker can push messages back to the ESP32 over this same connection at any time, without the ESP32 needing to accept any new inbound connections.

Your laptop opens its own separate outbound connection to the broker (from any connected network). The broker is the only entity that communicates with both devices. When the ESP32 publishes data such as a sensor reading, the broker receives it and immediately pushes it down the laptop's open connection. When the laptop publishes a command, the broker pushes it down the ESP32's connection. This is the same principle that allows your phone to receive push notifications on any network: the device reaches out, holds the connection, and the server pushes data through it.

Step-by-Step Setup

IMPORTANT: the setup process includes changing files on your ESP32. In particular, main.py will be overwritten, so if you have any code in that file that you want to retain be sure that your files are backed up on your laptop before performing the following steps.

NOTE: If your ESP32 is already set of for WiFi access through umd.iot, skip ahead to Step 4.

Step 1: Find your MAC address

Before your ESP32 can connect to campus WiFi, its wireless MAC address must be registered with UMD's network infrastructure. Each registered device receives a unique password. To find your ESP32 MAC address, download get_mac_address.py from the course Github repository, and use Thonny to run the code and display your ESP32 MAC address. The code will print something like MAC address: 24:6f:28:a1:b2:c3. Write this down for the next step.

Step 2: Register your ESP32 on the umd-iot network

For this step your computer must be on the campus network or connected to the campus VPN. Navigate to the UMD IoT device registration portal at https://it.umd.edu/connect. Log in with your university credentials, and select the *Gaming or Streaming Device* option. Select the Create Device link and register your ESP32 using the MAC address found above. The system will generate a device-specific password. Save this password — you will enter it in your ESP32's WiFi configuration.

> **Important:** The umd-iot network uses WPA2 with a per-device password. The network name (SSID) is "umd-iot" and the password is the one generated during registration. Do not use your university password.

Step 3: Prepare config.py, main.py, and wifi.py for WiFi and HiveMQ access

Download main.py, wifi.py, and config.py from Github and save each file to the ESP32. The main.py file should already exist on your ESP32, and will need to be overwritten for WiFi and HiveMQ access. Be sure to save any code from the existing file that you wish to retain.

The config.py file stores your WiFi and MQTT credentials. Be sure to change the values of TERPMAIL_USERNAME and UMD_IOT_PASSWORD. If you plan to work off campus using different WiFi networks, add a (ssid, password) pair for each network to the WIFI_NETWORKS list.

> **Important:** Never share your config.py or commit it to a public repository. It contains passwords.

Step 4: Install the MQTT library on the ESP32

Reboot your ESP32 to establish a working WiFi connection. Use the built-in mip (micro pip) MicroPython package manager to install the umqtt.simple library on the ESP32's filesystem from the Thonny REPL:

```
import mip
mip.install('umqtt.simple')
```

> **Note:** The mip package manager requires an active internet connection on the ESP32. If your ESP32 is not yet connected to WiFi, use the Thonny package manager instead: In Thonny, go to Tools > Manage packages. Search for "micropython-umqtt.simple" and click Install.

Step 5: Code example for HiveMQ connectivity

Download the example code (hivemq.py) that connects to the broker, subscribes to a topic for incoming messages, and publishes to other topics for outgoing data. Use mip to copy hivemq.py to your ESP32:

```
import mip
mip.install('github:ddevoe-
umd/enme441/micropython/10_Brokered_Communication/hivemq.py')
```

Note that the hivemq.py file will be placed in the /lib directory by mip, but this directory is in the standard MicroPython path for imports. If you would rather place the file in your root directory, add target='/' as a second argument to mip.install(). Keep in mind that mip.install() will overwrite any existing files with the same path and name.

Next, open the below file in a browser, and edit `config.py` on your ESP32 to add the new content at the bottom of this file. Don't change your existing WiFi password or network entries:

```
https://github.com/ddevoe-umd/enme441/blob/main/micropython/10_Brokered_Communication/config-dot-py_edits.py
```

Be sure to edit the entry for TERPMAIL_USERNAME to contain your actual terpmail username – this will be included in MQTT topic strings to differentiate your communication with the broker from other users.

Now that you are set up, take a look at the hivemq.py code file to see how this code works:

`on_message(topic, msg)` is a callback function that fires whenever a message arrives on a subscribed topic. It parses the incoming JSON and acts on recognized commands. You will extend this function as you add more hardware to your project.

`start_comms()` creates an MQTT client connected with the HiveMQ server, subscribes the client to check for messages from the server on specified topics (including messages initiated from your laptop), and publishes data to the server.

The main loop starts the communications (once) and then continually calls client.check_msg() to process any pending incoming messages, and publish random "sensor" values as JSON to the sensor topic. The repeated check_msg() calls in the infinite loop prevent the TCP keep-alive from timing out.

> **Important:** Do not use long time.sleep() calls (more than 30 seconds) in the main loop. The MQTT connection has a keep-alive interval, and if your code blocks for too long without calling check_msg() or ping(), the broker will consider the client dead and disconnect it.

Step 6: Test from your laptop

First, run the hivemq.py code on your ESP32:

```
import hivemq
hivemq.run()
```

You should see dictionary data being periodically sent from the ESP32 to the HiveMQ broker. Note that if you want this code to run at startup, you will need to add the above 2 lines to main.py.

With the code running, use your laptop to monitor and interact with your ESP32 using MQTT Explorer. Download and install MQTT Explorer for your laptop from https://mqtt-explorer.com. Open MQTT Explorer and create a new connection with the following settings:

- **Name:** (arbitrary)
- **Host:** 69702dc0158e4b3ea7e406ed4c78ae13.s1.eu.hivemq.cloud
- **Port:** 8883
- **Username / Password:** enme441 / enme441iot (shared MQTT credentials)
- **Encryption (TLS):** enabled

After connecting, you will see the topic tree on the left. Expand enme441 > your_terpmail_username and you should see sensor messages arriving every 5 seconds.

To send a command back to your ESP32, use the Publish panel at the bottom. Set the topic to

```
enme441/your_terpmail_username/commands
```

and set the payload to:

```
{"cmd": "LED_ON"}
```

Click Publish. The ESP32's built-in LED should turn on. Now try turning the LED off:

```
{"cmd": "LED_OFF"}
```

Rather than using MQTT Explorer, you could instead use command-line tools (mosquitto_sub and mosquitto_pub) or write a Python script (using the paho-mqtt library) to subscribe to your sensor topic (to see incoming data) and publish to your command topic (to control your ESP32).

Congratulations, your ESP32 is now set up for use as a flexible and adaptable microcontroller for broker-based IoT communications!

Troubleshooting

WiFi connection issues

- **"Connecting to WiFi..." hangs indefinitely:** Verify that the MAC address registered with UMD matches your ESP32 exactly. Re-run the MAC address command to double-check. Also confirm you are using the device password from the IoT registration portal, not your university password.
- **WiFi connects but no internet:** The umd-iot network may take a few seconds to fully provision a new device. Try resetting the ESP32. If it persists, verify your registration is still active on the UMD portal.

MQTT connection issues

- **"ECONNABORTED" error:** The MQTT server can become dormant after a period of inactivity. Try reconnecting if the initial connection fails.
- **"MQTTException" or connection refused:** Check that you are using port 8883 (TLS), not 1883 (unencrypted). Verify your MQTT username and password. Make sure the ssl=True parameter is set in the MQTTClient constructor.
- **"OSSL" or SSL/TLS errors:** Ensure your MicroPython firmware is a recent stable release. Older firmware versions may not support the TLS version required by HiveMQ Cloud. Also verify that the server_hostname in ssl_params matches the broker URL exactly.
- **Connection drops after 60–90 seconds:** Your main loop is probably blocking too long between check_msg() calls. Reduce your sleep interval or include a keepalive parameter in the MQTTClient constructor.

Message issues

- **ESP32 publishes but laptop doesn't receive:** Confirm the laptop is subscribing to the exact same topic the ESP32 is publishing to. Topics are case-sensitive and must match character for character.
- **Laptop publishes command but ESP32 doesn't react:** Make sure the ESP32 is calling client.check_msg() in its main loop. Without this call, incoming messages are never processed. Also verify the on_message callback is correctly parsing the JSON payload.
- **Messages appear garbled or truncated:** Ensure you are publishing valid JSON. Use json.dumps() on the sending side and json.loads() on the receiving side. Avoid publishing raw strings that look like JSON but have mismatched quotes.

General debugging tips

- **Use Thonny's REPL:** When the ESP32 is connected via USB, all print() statements appear in the Thonny console. This is the fastest way to debug connection and message issues.
- **Use MQTT Explorer:** Connect MQTT Explorer to the broker and watch the live topic tree. You can see every message being published and verify topics and payloads in real time.
- **Test in isolation:** Before combining WiFi + MQTT + sensors, test each piece separately. Verify WiFi connects, then verify MQTT connects, then add sensor reading last.